Checkpoint D - Architecture and Changeability

2026-11-12

Architecture

Service-Based, domain-partitioned, six services, one image:

Service	Capability
gateway	HTTP boundary, static client, profile, upload
data	ingestion, content identity, timeframe ladder, catalog
model	readings, states, events, progression
backtest	orchestration, strategy, execution, portfolio
evidence	provenance, digest, run records, memory
simulation	windows, marks, oracle, review, check

The pipeline is synchronous; the run lifecycle is asynchronous; research is a separate, isolated plane. Stages exchange content references, never payload sets. Database, object store, and queue sit behind traits, so a second implementation is additive.

Decisions

Twelve recorded decisions. The two that are hard to reverse:

Decision	Why hard to reverse	Revisit when
Canonical digest definition	Invalidates every recorded comparison	a digest misses a real artefact class
Run identity inputs	Invalidates deduplication and every run id	a new input changes results

Quantified characteristics

Characteristic	Level
Testability	every reading and event has a causality, a determinism, and a golden test; fast suite <= 5 min

Characteristic	Level
Modularity	6 services, 0 cycles, gateway imports 0 engine symbols
Maintainability	a rule change touches <= 1 crate and <= 3 modules; no file over 1,000 lines
Deployability	clean clone to healthy <= 10 min; 1 published port
Fault tolerance	0 succeeded runs with a missing artifact; resubmission gives identical bytes
Responsiveness	submit <= 200 ms; chart window <= 200 ms; 1 year over 8 timeframes <= 60 s on 8 cores
Scalability	model stage 1 to 4 workers: identical digest, >= 2.5x throughput
Partitioning type	domain-driven: 6 services by capability, no layer split

Changeability evidence

Change	Ripple	Cost	What stayed easy
Add a service	2 decisions, 9 work items	one documentation pass	the engines were untouched
Add profiles	one column in four schemas, one header, one client store	small	no service changed its responsibility
Add a reading	one module, registry, parser, writer, one fixture	the widest ripple in the system	bounded to one crate

Cheap to change: a rule, a reading, a client screen, an endpoint behind a version. Expensive: the digest definition, the run identity, the service count.

Consistency

Eight requirements -> eight characteristics -> six services -> twelve decisions -> one work tree. One increment throughout: one rule in, one reproducible bundle out.

Likely questions

- **Cheap:** a rule change, a new reading, a client screen, a second worker.
- **Expensive:** the digest definition, the run identity, a seventh service.
- **Residual risk:** the distributed stack is partly built; the determinism claim is bounded to one machine.